Catching the Wind, Saving the Fuel

Nonlinear Model Predictive Control for Wind Assisted Ship Propulsion

By Hilmi

HILMI

Research Areas: Nonlinear Control · Maritime Decarbonization · Stochastic Optimization

Key Methods: Model Predictive Control · Economic MPC · Receding Horizon Optimization

Applications: Ship Routing · Energy Management · Sustainable Shipping

Project Statement

My research sits at the intersection of **mathematical optimization**, **applied machine learning**, and **real world decision systems**. I am motivated by a single question: *how do we design algorithms that are theoretically sound, computationally efficient, and operationally interpretable, all at once?*

Wind assisted shipping is a near perfect testbed for this question. The International Maritime Organization has set a target of cutting shipping emissions by 50 percent by 2050, and Flettner rotor retrofits are one of the very few technologies that can deliver double digit fuel savings on existing vessels. Yet the rotors only help *when the captain knows how to use them.* Spin them too aggressively in headwind and they become a drag source. Leave them idle in beam wind and the ship burns fuel it did not need to burn. Deciding the right spin ratio, engine RPM, and cruise speed at every moment of a voyage is a constrained nonlinear optimization problem under uncertainty. This is exactly the kind of problem that model predictive control was designed to solve.

Why this project matters beyond shipping. The core computational pattern, solve a small nonlinear program every few seconds using a short lookahead horizon and a rolling forecast, appears in almost every cyber physical system. Autonomous vehicles choose steering and throttle the same way. Power grids choose generator dispatch the same way. Emergency service fleets choose ambulance repositioning the same way. My earlier work on reinforcement learning for ambulance redeployment and on optimal transport for distribution matching already crossed paths with these ideas; this project consolidates them into a single, coherent control theoretic framework that I can carry into doctoral research.

What I learned through careful numerical auditing. Perhaps the most valuable outcome was realizing, partway through the project, that my naive always on rotor baseline was *losing* fuel 11 percent of the time. My first instinct was to treat this as a bug. After re reading Tillig and Ringsberg (2020), I understood it was a physical phenomenon: in headwind configurations the rotor's drag coefficient dominates its lift coefficient, and the ship spends propeller power fighting its own sail. That accidental discovery became the single strongest argument in the entire project for why intelligent closed loop control, rather than static rules, is the right answer. Similar care, checking units, verifying theoretical invariants, re tuning weights when a controller produced physically implausible speeds, was needed throughout the implementation. This experience reinforced my conviction that **reproducible research requires more than reproducible code**: it requires someone who understands the underlying physics well enough to know when the numbers are wrong.

Broader trajectory. I see this project as one step in a larger research agenda: building mathematical tools that bring rigor to sustainability problems while remaining transparent and deployable on real hardware. Maritime transport is responsible for nearly 3 percent of global CO_2 emissions. If even a fraction of the world's fleet adopted MPC based energy management on top of wind propulsion retrofits, the cumulative fuel savings would be measured in megatons per year. That is the scale of impact I want my tools to contribute to.

Contents

1 The Business Problem: Why Intelligent Wind Assisted Shipping Matters **5**
1.1 The Maritime Emission Crisis . . . 5
1.2 The Captain's Dilemma . . . 5
1.3 Why Classical Rule Based Control Is Not Enough . . . 6
1.4 Beyond Shipping: Where This Framework Applies . . . 6
1.5 Quantifying the Impact . . . 6

2 Mathematical Foundations **7**
2.1 Ship Dynamics: The Maneuvering Model . . . 7
2.2 Propeller Thrust . . . 8
2.3 Flettner Rotor Aerodynamics: The Magnus Effect . . . 8
2.4 Apparent Wind Computation . . . 9
2.5 Stochastic Wind Modeling . . . 9
2.6 Model Predictive Control: The Receding Horizon Principle . . . 10
2.7 Two MPC Formulations: Tracking and Economic . . . 10
2.7.1 Tracking MPC . . . 11
2.7.2 Economic MPC . . . 11
2.8 Forecast Injection via Time Varying Parameters . . . 11

3 Project Architecture **12**
3.1 Overview of the Seven Files . . . 12
3.2 File 1: `src/ship_dynamics.py` . . . 12
3.3 File 2: `src/wind_field.py` . . . 13
3.4 File 3: `src/pid_baseline.py` . . . 13
3.5 File 4: `src/mpc_controllers.py` . . . 13
3.6 File 5: `src/simulator.py` . . . 14
3.7 File 6: `src/plotting.py` . . . 14
3.8 File 7: `experiments/run_single.py` . . . 15
3.9 File 8: `experiments/run_monte_carlo.py` . . . 15

4 Operating Conditions and Data **15**
4.1 The Reference Vessel: A Mid Sized Bulker with Two Flettner Rotors . . . 15
4.2 The Reference Voyage Profile . . . 16
4.3 The Wind Condition Ensemble . . . 16
4.4 Visualizing the Ensemble . . . 16

5 Experiments and Results **17**
5.1 Experiment 1: Single Scenario Head to Head Comparison . . . 17
5.2 Experiment 2: Monte Carlo Sweep Across Wind Conditions . . . 18

6 Visualizations **19**
6.1 Figure 1: Single Scenario Four Panel Comparison . . . 20
6.2 Figure 2: Monte Carlo Summary . . . 21
6.3 Figure 3: Wind Ensemble . . . 23

7 The Road to Correct Results: A Debugging Story **23**

HLMI

7.1 Mistake 1: Fuel Consumption Was Approximately 3600 Times Too High 24
7.2 Mistake 2: Economic MPC Behaved Just Like Tracking MPC 24
7.3 Mistake 3: Tracking MPC Barely Beat the PID Baseline 25
7.4 Mistake 4: The Frozen Plant Forecast Was Too Long 25
7.5 The Finding that Looked Like a Bug: Fixed Sail Loses on Average 26

8 Reproducing the Results: A Step by Step Guide 26
8.1 Requirements 26
8.2 Running the Pipeline 26
8.3 Suggested Exercises for the Reader 27

9 Limitations and Future Directions 27
9.1 Current Limitations 27
9.2 Future Directions 28

HILMI

1 The Business Problem: Why Intelligent Wind Assisted Shipping Matters

1.1 The Maritime Emission Crisis

International shipping moves roughly 90 percent of world trade by volume and is responsible for nearly 3 percent of global CO_2 emissions (Faber et al., 2020). If the industry were a country, it would rank as the world's sixth largest polluter, ahead of Germany. The International Maritime Organization (IMO) has set targets to cut absolute greenhouse gas emissions by at least 50 percent by 2050 compared to 2008 levels, with an aspirational goal of reaching net zero. Meeting these targets requires a portfolio of measures: new fuels (ammonia, methanol, hydrogen), hull retrofits, operational optimization, and wind propulsion.

Wind propulsion is particularly attractive because it works *with existing ships*. A commercial vessel with a 25 year service life cannot be scrapped and rebuilt; it can, however, be retrofitted with Flettner rotors, rigid wing sails, or soft kites in a few weeks at dry dock. Publicly reported sea trials on vessels equipped with Norsepower rotors, including the MV Maersk Pelican and the MV Annika Braren, have demonstrated fuel savings in the range of 5 to 15 percent on favorable routes (Norsepower, 2019). For an industry that runs on single digit margins, this is transformative economics.

1.2 The Captain's Dilemma

Fitting a rotor is the easy part. Using it well is the hard part. Every second of a voyage, the captain (or increasingly, the captain's software) must decide three things:

1. **Engine load**: How fast should the propeller turn?
2. **Sail state**: Should the rotor spin? At what spin ratio?
3. **Speed target**: Is it worth slowing down to burn less fuel per nautical mile, even if it means arriving late?

These three decisions are coupled. Spinning the rotor in beam wind produces forward thrust, so the engine can throttle back. Spinning the rotor in headwind produces drag, so the engine must work harder to maintain speed. Slowing the ship reduces fuel burn nonlinearly (roughly with the cube of speed) but may violate the scheduled arrival time and incur contractual penalties. The optimal decision depends on the wind forecast for the *next several minutes*, the current ship state, and the schedule constraints, all at once.

The Core Question of This Project

Can we design a controller that takes short term wind forecasts as input, looks a few minutes into the future, and produces control commands for the engine and the rotor that minimize fuel consumption while respecting speed and schedule constraints? The answer, as we demonstrate, is **yes**, via nonlinear model predictive control with a receding horizon.

HLMI

1.3 Why Classical Rule Based Control Is Not Enough

Before we built the MPC, we asked a reasonable question: is closed loop optimization actually necessary? A simpler alternative would be to run the ship at a constant reference speed using a PID governor on the engine, and switch the rotor on whenever the wind exceeds some threshold. This is how most ships operate today. We tested this baseline thoroughly in our experiments (Section 5) and discovered something striking: *a naive always on rotor policy loses fuel on average, compared to leaving the rotor off entirely.*

The reason, documented in Tillig and Ringsberg (2020) and Lu and Ringsberg (2020), is that the Flettner rotor's drag coefficient grows roughly linearly with spin ratio. When the apparent wind direction points close to the bow (headwind), the drag component dominates the lift component, and the rotor becomes a liability rather than an asset. A static on off rule cannot know this without measuring the apparent wind direction, reasoning about what the ship will do in the next minute, and potentially lowering the spin ratio or switching the rotor off.

1.4 Beyond Shipping: Where This Framework Applies

The optimization pattern we develop here, minimize a running fuel cost subject to dynamics and constraints, using a rolling short horizon forecast, appears in many other domains:

1. **Hybrid Electric Vehicles:** The engine and the battery play the roles of the propeller and the rotor. The MPC decides when to charge, when to discharge, and when to run the engine.
2. **Building Energy Management:** Heating, ventilation, and air conditioning systems must track a comfort setpoint while exploiting solar gain and electricity price fluctuations.
3. **Power Grid Dispatch:** The grid operator decides which generators to run minute by minute, balancing renewable generation, demand, and storage.
4. **Autonomous Driving:** Steering, throttle, and brake commands are chosen to track a reference trajectory while respecting vehicle dynamics and obstacle constraints.
5. **Emergency Fleet Deployment:** My earlier work on ambulance repositioning under stochastic demand shares the same mathematical structure: short horizon decisions under uncertainty about future events.

The methods we build for a wind assisted bulker transfer, almost verbatim, to any of these settings. This is why model predictive control has become the dominant industrial control methodology in the last two decades.

1.5 Quantifying the Impact

Table 1 summarizes the headline result of this project, averaged across 12 wind scenarios (a Monte Carlo ensemble spanning favorable, beam, and headwind conditions). The full

HLLM

experimental setup is described in Section 5.

Table 1: Monte Carlo average performance across 12 wind scenarios.

Controller	Fuel (kg/nm)	Speed (kn)	Savings	Solve time
PID engine only	44.57	12.03	baseline	0 ms
PID with always on sail	49.55	12.03	−11.2%	0 ms
Tracking NMPC	43.40	11.97	+2.6%	8 ms
Economic NMPC	**35.04**	10.29	+21.4%	13 ms

Two messages stand out. First, the naive always on sail baseline *worsens* fuel economy because it does not account for wind direction. Second, the economic MPC delivers a 21 percent fuel reduction by exploiting both optimal rotor scheduling and modest slow steaming, at a per step compute cost of 13 milliseconds, which is three orders of magnitude below the 10 second control period. This means the method is real time feasible on commodity shipboard hardware.

2 Mathematical Foundations

HLMI

This section develops the theory from first principles. We begin with ship dynamics, progress through Flettner rotor aerodynamics and wind modeling, introduce model predictive control, and arrive at the two MPC formulations used in the project.

2.1 Ship Dynamics: The Maneuvering Model

We model the ship as a rigid body moving in three degrees of freedom on the water surface: surge (forward), sway (sideways), and yaw (rotation about the vertical axis). Heave, roll, and pitch are neglected, which is standard for control design on cargo vessels in moderate sea states (Fossen, 2011).

Definition 2.1 (Ship State and Inputs)**.** The ship state at time t is the vector

$$\mathbf{x}(t) = (X, Y, \psi, u, v, r)^\top \in \mathbb{R}^6,$$

where (X, Y) are the position in an earth fixed frame, ψ is the heading angle, and (u, v, r) are the surge, sway, and yaw velocities in the body fixed frame. The control input is

$$\mathbf{u}(t) = (n, \delta, SR)^\top \in \mathbb{R}^3,$$

where n is the propeller shaft speed in revolutions per second, δ is the rudder angle in radians, and SR is the rotor spin ratio, a dimensionless scaling between rotor rotational speed and apparent wind speed.

The equations of motion follow the classical Fossen formulation:

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\,\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\,\boldsymbol{\nu} \;=\; \boldsymbol{\tau}_{\text{prop}}(n, u) + \boldsymbol{\tau}_{\text{rudder}}(\delta) + \boldsymbol{\tau}_{\text{rotor}}(SR, V_{\text{aw}}, \beta_{\text{aw}}), \tag{1}$$

where $\boldsymbol{\nu} = (u, v, r)^\top$ is the body frame velocity, $\mathbf{M}$ combines rigid body inertia and hydrodynamic added mass, $\mathbf{C}$ is the Coriolis and centripetal matrix, $\mathbf{D}$ represents hydrodynamic damping, and the right hand side collects the three control forces.

Why a Simplified Three Degree of Freedom Model Is Enough

A full six degree of freedom simulator would include heave, roll, and pitch, plus coupling with wave loading. For control *design*, however, these modes are fast (wave induced) and decoupled from the slow maneuvering dynamics we care about. Fossen (2011) and Perez and Fossen (2011) make a standard argument: controllers designed on the three degree of freedom model transfer cleanly to full scale ships, provided the unmodeled dynamics are treated as disturbances.

2.2 Propeller Thrust

The propeller delivers a thrust T_{prop} aligned with the surge axis:

$$T_{\text{prop}}(n, u) \;=\; (1 - t_{\text{prop}})\, \rho_w\, K_T\, n^2\, D_{\text{prop}}^4, \tag{2}$$

where t_{prop} is the thrust deduction factor (a correction for hull wake interaction), ρ_w is seawater density, K_T is the dimensionless thrust coefficient, and D_{prop} is the propeller diameter. We use a quasi constant K_T model, which is accurate within a few percent at service speed operating points.

The shaft power required is $P_{\text{shaft}} = T_{\text{prop}} \cdot V_a/\eta_D$, where $V_a = u(1 - w_{\text{prop}})$ is the advance velocity (surge speed corrected for wake) and η_D is the propulsive efficiency. The fuel consumption rate is then

$$\dot{m}_{\text{fuel}}(n, u) \;=\; \text{SFC} \cdot P_{\text{shaft}}(n, u), \tag{3}$$

where SFC is the specific fuel consumption of the main engine, typically around 180 grams per kilowatt hour for a modern two stroke marine diesel.

2.3 Flettner Rotor Aerodynamics: The Magnus Effect

A Flettner rotor is a vertical cylinder that spins about its own axis. When apparent wind flows past a spinning cylinder, the Magnus effect produces a force perpendicular to the wind vector. This was first described by Magnus (1852) in the context of ballistics, and applied to ships by Anton Flettner in the 1920s.

The dimensionless spin ratio controls the aerodynamic behavior:

$$SR \;=\; \frac{\omega_{\text{rotor}}\, r_{\text{rotor}}}{V_{\text{aw}}}, \tag{4}$$

where ω_{rotor} is the rotor angular velocity, r_{rotor} is the rotor radius, and V_{aw} is the apparent wind speed at the rotor. A spinning rotor produces a lift coefficient $C_L(SR)$ and a drag

coefficient $C_D(SR)$. Based on full scale measurements fitted in Tillig and Ringsberg (2020), we use the smooth approximations

$$C_L(SR) = 10\left(1 - e^{-0.55\,SR}\right), \qquad C_D(SR) = 0.5 + 0.35\,SR. \tag{5}$$

The lift saturates near $C_L \approx 10$ (much higher than a conventional sail, which plateaus near $C_L \approx 1.5$), while drag grows roughly linearly. At low spin ratios, the lift to drag ratio is high and the rotor is efficient. At high spin ratios, the drag penalty becomes substantial.

Why Drag Matters for Fuel Economy

At a spin ratio of 5 (near the physical limit of most commercial rotors) in apparent wind of 10 meters per second, each rotor produces roughly 70 kilonewtons of lift. If the apparent wind points broadly sideways, this lift projects forward onto the ship and provides free thrust. If the apparent wind points toward the bow, the *drag* component dominates the projection, and the rotor now pushes the ship *backward*. The magnitude of this adverse effect is roughly $\frac{1}{2}\rho_a A_{\text{rot}} V_{\text{aw}}^2 C_D(SR)$, which for our parameters is tens of kilonewtons. This is the physical mechanism behind the negative Monte Carlo savings of the always on sail baseline.

2.4 Apparent Wind Computation

The *true wind* is what a stationary weather buoy would measure. The *apparent wind* is what the moving ship feels, obtained by subtracting the ship's velocity from the true wind vector:

$$\mathbf{V}_{\text{aw}}^{\text{earth}} = \mathbf{V}_{\text{tw}}^{\text{earth}} - \mathbf{V}_{\text{ship}}^{\text{earth}}, \tag{6}$$

where the ship's velocity in the earth frame is obtained from the body frame velocity by a rotation through the heading angle ψ. Rotating the apparent wind vector back into the body frame gives the apparent wind speed V_{aw} and the angle of attack β_{aw} (measured from the bow, positive to starboard). The rotor forces resolve into surge and sway components that depend on both V_{aw} and β_{aw}.

2.5 Stochastic Wind Modeling

Real maritime wind has structure at multiple time scales. Over hours, a synoptic weather system passes through and shifts the mean wind speed and direction. Over minutes, mesoscale gusts fluctuate around the synoptic mean. Over seconds, turbulence causes high frequency jitter. For control design at a 10 second sampling rate, we capture the first two scales using a two component Ornstein Uhlenbeck (OU) process (Uhlenbeck and Ornstein, 1930):

$$dX_t = -\frac{1}{\tau}(X_t - \mu)\,dt + \sigma\sqrt{\frac{2}{\tau}}\,dW_t. \tag{7}$$

The OU process is the continuous time analog of an autoregressive process of order 1. It has three parameters: the long run mean μ, the correlation time τ, and the stationary standard deviation σ. Summing a slow component ($\tau \approx 1$ hour) and a fast component ($\tau \approx 5$ minutes) for both wind speed and direction yields realistic time histories.

Why Ornstein Uhlenbeck and Not Just White Noise

White noise would produce wind fluctuations that are uncorrelated across time, which is unphysical: the wind speed 10 seconds from now is overwhelmingly similar to the wind speed right now. The OU process is the simplest stochastic model that is Gaussian, stationary, and Markovian, while still reproducing the autocorrelation observed in real anemometer data. The classical reference is the Dryden turbulence model used in aerospace, whose low frequency component is essentially the OU process we use here.

2.6 Model Predictive Control: The Receding Horizon Principle

Model predictive control solves a constrained optimization problem at every control step, using a model of the system to predict the evolution of the state over a short lookahead window, called the prediction horizon. Only the first control action is applied; then the horizon shifts forward and the optimization is repeated. This is the *receding horizon principle* (Mayne et al., 2000; Rawlings et al., 2017).

Formally, at each sampling instant k we solve

$$\min_{\{\mathbf{u}_{k|k},\ldots,\mathbf{u}_{k+N-1|k}\}} \sum_{j=0}^{N-1} \ell(\mathbf{x}_{k+j|k}, \mathbf{u}_{k+j|k}) + V_f(\mathbf{x}_{k+N|k}), \tag{8}$$

subject to the dynamics $\mathbf{x}_{k+j+1|k} = f(\mathbf{x}_{k+j|k}, \mathbf{u}_{k+j|k}, \mathbf{w}_{k+j|k})$, state constraints $\mathbf{x} \in \mathcal{X}$, and input constraints $\mathbf{u} \in \mathcal{U}$, with initial condition $\mathbf{x}_{k|k} = \mathbf{x}(t_k)$. Here N is the horizon length, ℓ is the stage cost, V_f is a terminal cost, and $\mathbf{w}_{k+j|k}$ is the forecast of the disturbance (the wind) at step $k+j$ as seen from time k.

Why MPC and Not Just a PID Controller

A PID controller tracks a setpoint by reacting to the present tracking error. It cannot anticipate. If the wind shifts in 30 seconds, a PID controller only learns about it after the error has already built up. An MPC controller, given a short forecast, can act *now* to prepare for the change. This is especially valuable for nonminimum phase systems like ships, where large inertia means actions taken today only show their effect many seconds later. Predictive controllers outperform reactive ones whenever the dominant time scale of the system exceeds the sampling period, which is always true for ships.

2.7 Two MPC Formulations: Tracking and Economic

We implement two distinct MPC controllers that share the same internal model but differ in their cost functions.

2.7.1 Tracking MPC

The tracking MPC is the classical setpoint formulation. It tries to hold a reference speed u_{ref} while using the rotor to reduce fuel burn as a secondary objective:

$$\ell_{\text{track}}(u, n, SR) \;=\; w_{\text{track}}\,(u - u_{\text{ref}})^2 + w_{\text{fuel}}\,\dot{m}_{\text{fuel}}(n, u). \tag{9}$$

The weight w_{track} is large relative to w_{fuel}, so the primary behavior is speed holding. When the rotor can provide forward thrust (favorable wind), the MPC throttles the engine down to save fuel. When the rotor provides drag (headwind), the MPC keeps the rotor off or at low spin ratio and maintains engine load.

2.7.2 Economic MPC

The economic MPC (Ellis et al., 2014; Angeli et al., 2012) inverts the priority: fuel minimization is the primary objective, and schedule adherence is a soft penalty:

$$\ell_{\text{econ}}(u, n, SR) \;=\; w_{\text{fuel}}\,\dot{m}_{\text{fuel}}(n, u) + w_{\text{sched}}\,(u - u_{\text{sched}})^2. \tag{10}$$

A hard constraint $0.85\,u_{\text{sched}} \le u \le 1.10\,u_{\text{sched}}$ prevents the controller from slowing the ship to a crawl, which would defeat the purpose of the voyage. This soft penalty plus hard constraint structure is standard in commercial slow steaming practice (Psaraftis and Kontovas, 2013).

Tracking vs. Economic: The Underlying Philosophical Difference

Tracking MPC says "hold this speed, save fuel as a bonus." Economic MPC says "save fuel, hold speed within a comfort band as a safety." In favorable wind both controllers behave similarly: the tracking MPC can reduce engine load while holding speed, and the economic MPC has no reason to slow down. In unfavorable wind they diverge dramatically: the tracking MPC must push harder on the engine to maintain speed, while the economic MPC can choose to slow the ship and let the voyage take a little longer. This is the key mechanism behind the 21 percent Monte Carlo savings of the economic MPC.

2.8 Forecast Injection via Time Varying Parameters

Both controllers use a simplified one state internal model (surge dynamics only), since the lateral and yaw dynamics can be handled by a decoupled rudder autopilot. The apparent wind over the prediction horizon is passed to the optimizer as a *time varying parameter* (TVP) sequence. In deployment this would come from an onboard weather forecast stream. In our simulations it is evaluated from the true wind scenario along the ship's current heading, under the assumption that heading and speed stay approximately constant over the horizon (the *frozen plant* approximation, standard in voyage optimization).

The TVP injection happens at every control step: before calling the solver, we update a persistent buffer that the optimizer reads during its symbolic evaluation. This is the

idiomatic pattern in the `do-mpc` framework (Fiedler et al., 2023) we use for implementation.

3 Project Architecture

The project is organized as a self contained Python package with a strict separation between reusable library code (under `src/`) and runnable experiments (under `experiments/`). Each file has one clearly defined responsibility.

3.1 Overview of the Seven Files

The data flow is straightforward. The top level experiment scripts read a wind scenario from the generator, instantiate a plant and a controller, and run the closed loop simulator. Results are written as CSV and PNG files.

Library (under `src/`):
`ship_dynamics.py`, `wind_field.py`, `pid_baseline.py`, `mpc_controllers.py`, `simulator.py`, `plotting.py`

Experiments (under `experiments/`):
`run_single.py`, `run_monte_carlo.py`

3.2 File 1: `src/ship_dynamics.py`

This file defines the 3 degree of freedom maneuvering plant and the Flettner rotor aerodynamic model. It exposes two mathematically identical implementations: a NumPy version used for plant simulation (fast, differentiable by hand) and a CasADi symbolic version used inside the MPC optimizer (symbolic, automatically differentiable).

Key objects:

- `SHIP_PARAMS`: dictionary of vessel parameters (mass, inertia, added mass, damping coefficients, propeller geometry, rotor dimensions, engine parameters, SFC).
- `apparent_wind_np(u, v, psi, V_tw, psi_tw)`: computes apparent wind speed and angle of attack from the true wind and the ship state.
- `rotor_forces_np(V_aw, beta_aw, SR, p)`: resolves the lift and drag forces into body frame surge and sway components.
- `rhs_numpy(state, inputs, wind, p)`: evaluates $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, \mathbf{w})$ for plant simulation.
- `rk4_step(state, inputs, wind, dt, p)`: fourth order Runge Kutta integrator wrapping the right hand side.
- `fuel_rate_np` and `fuel_rate_ca`: fuel burn rate in kilograms per second, for simulation and for MPC respectively.

3.3 File 2: `src/wind_field.py`

This file constructs the stochastic wind time histories used as environmental forcing. Wind scenarios are produced by a two scale Ornstein Uhlenbeck process (Equation 7), combining a slow synoptic component (correlation time approximately one hour) with a faster gust component (correlation time approximately five minutes). Both wind speed and wind direction are modeled independently as OU processes.

Key objects:

- `WindScenario`: a dataclass holding the time grid, wind speed array, and wind direction array. Supports interpolation via the call operator: `V, psi = scenario(t)`.
- `generate_scenario(...)`: produces one wind realization with specified mean, variance, correlation time, and random seed.
- `ensemble(n_scenarios, base_seed, ...)`: generator yielding a deterministic ensemble of scenarios, used for Monte Carlo.

3.4 File 3: `src/pid_baseline.py`

This file implements the two baseline controllers that MPC is compared against.

- `PIDSpeedController`: a classical PID regulator on surge speed with gain scheduling and anti windup.
- `PDHeadingAutopilot`: a proportional derivative autopilot on heading, used by every controller to steer the ship on a straight reference course. This decouples lateral dynamics from the longitudinal MPC.
- `fixed_sail_policy(V_tw, mode, SR_nom, V_on)`: returns a spin ratio based on a simple static rule. Two modes are implemented: `off` (rotor never spins) and `always_on` (rotor spins at $SR_{\text{nom}} = 3$ whenever the true wind exceeds a threshold).

3.5 File 4: `src/mpc_controllers.py`

This is the core algorithmic file. It constructs both MPC controllers using the `do-mpc` framework. Both controllers share an internal one state surge dynamics model (Section 2) built once in CasADi.

Key objects:

- `_build_internal_model(p)`: assembles the CasADi symbolic surge model with states, inputs, time varying parameters, the right hand side, and auxiliary expressions (fuel rate, rotor thrust, propeller thrust).
- `build_tracking_mpc(u_ref, t_step, horizon, w_track, w_fuel, ...)`: constructs the tracking MPC with the cost function in Equation 9. Returns a configured `do_mpc.controller.MPC` instance ready to call `make_step()`.

HLMI

- `build_economic_mpc(u_sched, t_step, horizon, w_fuel, w_sched, ...)`: constructs the economic MPC with the cost function in Equation 10. Hard speed constraints are set at 85 and 110 percent of the scheduled speed.

- `set_wind_forecast(mpc, V_aw_traj, beta_aw_traj)`: writes a fresh forecast trajectory into the MPC's TVP buffer before each call to `make_step()`.

The underlying numerical solver is IPOPT (Wächter and Biegler, 2006), called through CasADi's symbolic framework (Andersson et al., 2019).

3.6 File 5: `src/simulator.py`

This file contains the closed loop simulation engine that ties the plant, the controller, and the wind scenario together. It is responsible for the time stepping, the forecast injection, and the logging.

Key objects:

- `SimResult`: a dataclass holding all logged quantities from one closed loop run (time series of state, inputs, wind, fuel, solve times) and convenience properties for summary metrics (`distance`, `fuel_per_nm`, `avg_speed_kn`).

- `simulate(controller, wind_scenario, ...)`: the main entry point. Takes a controller (either a baseline dictionary or an MPC instance) and a wind scenario, runs the closed loop for the specified duration, and returns a `SimResult`. Internally it uses a 1 second RK4 plant integrator with a 10 second control sampling period, constructing the apparent wind forecast over the MPC horizon at each control instant.

3.7 File 6: `src/plotting.py`

This file generates publication quality figures from `SimResult` objects and the Monte Carlo data frame. It uses matplotlib with consistent styling: colorblind friendly palette, high DPI, clear titles and axis labels.

Key objects:

- `plot_single_scenario(results, savepath, u_ref)`: four panel figure showing environmental forcing, speed tracking, actuator commands, and cumulative fuel for all controllers on a single wind realization.

- `plot_monte_carlo(mc_df, savepath)`: two panel figure with box plots of fuel intensity and per scenario savings.

- `plot_wind_ensemble(scenarios, savepath)`: visualization of the wind time histories across the Monte Carlo ensemble.

HIMI

3.8 File 7: `experiments/run_single.py`

This is the first of two experiment scripts. It runs all four controllers on the same single wind realization to produce a head to head comparison. The workflow is: generate one wind scenario with a fixed seed, build the two MPC controllers, loop over the four controllers, simulate each one for the specified horizon, print a summary table, and save the four panel figure.

3.9 File 8: `experiments/run_monte_carlo.py`

This script runs the Monte Carlo sweep. Twelve wind scenarios are drawn from the ensemble (different mean directions uniformly over $[0, 2\pi]$ and different mean speeds uniformly over $[7, 14]$ meters per second). Every controller is evaluated on every scenario, producing $4 \times 12 = 48$ simulation runs. Results are aggregated into a pandas data frame and saved as CSV and PNG.

4 Operating Conditions and Data

HILMI

The study is carried out on a reference vessel profile and an ensemble of open sea wind conditions representative of North Atlantic cargo routes. This section describes both.

4.1 The Reference Vessel: A Mid Sized Bulker with Two Flettner Rotors

Parameters: `SHIP_PARAMS` dictionary in `src/ship_dynamics.py`

The simulated vessel is a representative 6,000 deadweight tonnage bulk carrier, the smallest size class that has been retrofitted with Flettner rotors in practice. Publicly documented examples include the MV Annika Braren (Rotortug B.V.) and research vessels studied in the NAVIGATOR and WindTech projects. Key parameters:

- **Mass:** 6,000 tonnes; yaw moment of inertia 1.2×10^{10} kg m^2.
- **Propeller:** 5 meter diameter, $K_T = 0.22$, shaft speed range 0.5 to 2.5 revolutions per second.
- **Engine:** Installed power 4 MW, specific fuel consumption 180 g/kWh.
- **Flettner rotors:** Two cylinders, each 30 meters tall by 4 meters in diameter, located 10 meters aft of the center of gravity.
- **Hydrodynamic coefficients:** Added mass, linear and quadratic damping, and Coriolis terms chosen consistent with published bulker geometry.

These parameters were chosen to match the operating envelope of a real vessel that could plausibly participate in the WINTEGRATE project or a similar retrofit program, and

are within the range of parameters used in the simulation studies of Tillig and Ringsberg (2020).

4.2 The Reference Voyage Profile

The vessel is assumed to be on a straight course transit, heading due north with a fixed heading reference. The schedule speed is 12 knots (approximately 6.2 meters per second), which is a typical service speed for Handysize and Supramax bulkers. Each experimental run simulates either 60 minutes (single scenario) or 20 minutes (Monte Carlo) of steady state operation, long enough for the controllers to reach their operating regime and for transient effects to wash out.

4.3 The Wind Condition Ensemble

Generator: `generate_scenario()` in `src/wind_field.py`

Wind conditions are drawn from an ensemble of 12 scenarios covering the range of situations a ship on open sea passage encounters. Each scenario is a time history of true wind speed and true wind direction at 10 second resolution. The statistical characteristics of this ensemble match observed marine wind behavior reported in classical aerospace and maritime literature (Uhlenbeck and Ornstein, 1930).

Speed distribution. Mean wind speeds across the ensemble are uniformly distributed between 7 and 14 meters per second, corresponding to Beaufort 4 (moderate breeze) through Beaufort 6 (strong breeze). These are the conditions under which Flettner rotors are most commonly deployed: below Beaufort 3, the wind is too weak to provide meaningful thrust; above Beaufort 7, wave loading becomes the dominant concern and routing often takes priority over rotor scheduling.

Direction distribution. Mean wind directions are uniformly distributed across all points of the compass. This is a deliberately challenging ensemble: it includes headwind scenarios (where the rotor is a liability), beam wind scenarios (where the rotor is most effective), and tailwind scenarios (where both the rotor and slow steaming favor the ship). The uniform coverage is what makes the Monte Carlo result a meaningful statement about average operational behavior.

Variability within each scenario. Each scenario has a slow drift component (standard deviation 2.0 meters per second, correlation time 1 hour) and a faster gust component (standard deviation 1.2 meters per second, correlation time 5 minutes). Wind direction varies around its mean with a slow standard deviation of 0.18 radians (about 10 degrees) and a gust standard deviation of 0.06 radians.

4.4 Visualizing the Ensemble

A visualization of the 12 wind scenarios is produced as part of the Monte Carlo pipeline and is saved as the file named in the box below. The wind speed panel shows the dispersion

between the low end (around 7 m/s) and the high end (around 14 m/s) scenarios, while the wind direction panel shows the spread across the full compass.

Figure file: `results/wind_ensemble.png`

Reader's note: the figure produced by the pipeline should be inserted here in the final document. The filename above refers to the actual file path in the project folder.

5 Experiments and Results

Two complementary experiments characterize the controllers: a single scenario head to head comparison, and a Monte Carlo sweep over the 12 wind conditions. All numerical results in this section come directly from the output of `run_single.py` and `run_monte_carlo.py`.

5.1 Experiment 1: Single Scenario Head to Head Comparison

Script: `experiments/run_single.py`
Figure: `results/single_scenario.png`
Console output: printed summary table

Goal. Run all four controllers on the same wind realization and observe how they behave in detail: how each controller trades off speed holding, rotor use, and engine load, and how these choices translate into fuel consumption.

Setup. A single wind scenario with mean wind speed 11 meters per second, mean direction 80 degrees (roughly beam from starboard), slow standard deviation 2.5 meters per second, gust standard deviation 1.2 meters per second, 60 minute duration, 10 second control period. Reference speed 12 knots. The tracking MPC uses a prediction horizon of 18 steps (3 minutes), with weights $w_{\text{track}} = 50$ and $w_{\text{fuel}} = 8$. The economic MPC uses a prediction horizon of 24 steps (4 minutes), with weights $w_{\text{fuel}} = 150$ and $w_{\text{sched}} = 3$.

Key findings. Table 2 summarizes the four controllers:

Table 2: Single scenario results: 60 minute leg in Beaufort 5 beam wind.

Controller	Fuel (kg)	Distance (nm)	Fuel per nm (kg/nm)	Savings
PID engine only	543.6	12.04	45.15	baseline
PID with fixed sail	447.8	12.05	37.18	+17.66%
Tracking NMPC	408.9	11.97	34.15	+24.37%
Economic NMPC	**278.9**	10.33	**27.01**	+40.19%

Three observations stand out:

- **The fixed sail baseline captures about 18 percent of the fuel savings.** This is a meaningful gain and is consistent with commercial sea trial reports. It shows

that even crude rotor scheduling helps when the wind is favorable. The beam wind in this scenario is almost ideal for Flettner rotors.

- **The tracking MPC adds 7 more percentage points (24 percent total) while still holding the 12 knot reference.** The extra savings come from smarter rotor use: the MPC spins the rotor up to the limit (SR = 5) whenever the apparent wind allows it, and reduces engine load accordingly. The speed tracking panel of the figure shows the tracking MPC holds 12 knots to within 0.2 knots throughout the run.
- **The economic MPC captures an additional 16 percentage points (40 percent total) by slow steaming to 10.3 knots.** This is slightly over the scheduled speed's hard constraint band (10.2 to 13.2 knots), which the controller respects exactly. The fuel curve flattens noticeably compared to the tracking MPC, showing that the economic controller's willingness to trade speed for fuel pays off in unfavorable moments within the scenario.

Solve time. Both MPCs return a control action in under 15 milliseconds on a laptop CPU, against a 10 second control period. This is a factor of 1000 safety margin for real time deployment on shipboard hardware.

5.2 Experiment 2: Monte Carlo Sweep Across Wind Conditions

Script: `experiments/run_monte_carlo.py`
Figure: `results/monte_carlo.png`
CSVs: `results/monte_carlo.csv`, `results/monte_carlo_summary.csv`

Goal. Quantify the *average* fuel savings of each controller across a realistic distribution of wind conditions, and measure the variability of those savings.

Setup. Twelve wind scenarios drawn as described in Section 4. Every controller is evaluated on every scenario, yielding 48 simulation runs. Each run is 20 minutes long; this is shorter than Experiment 1 but long enough for the controllers to reach their operating regimes, and keeps the total Monte Carlo runtime under 30 seconds on a laptop.

Key findings. Table 3 aggregates across all 12 scenarios:

Table 3: Monte Carlo results, averaged across 12 wind conditions.

Controller	Mean fuel/nm (kg/nm)	Std	Mean speed (kn)	Savings
PID engine only	44.57	0.48	12.03	baseline
PID with fixed sail	49.55	4.58	12.03	−11.18%
Tracking NMPC	43.40	1.97	11.97	+2.63%
Economic NMPC	**35.04**	2.48	10.29	+21.39%

The Monte Carlo numbers tell a different and more nuanced story than the single scenario results:

- **The fixed sail baseline worsens fuel economy by 11 percent on average.** In headwind scenarios (roughly a third of the ensemble), the rotor's drag coefficient

dominates and the ship burns extra fuel fighting its own sail. The standard deviation of 4.58 kg/nm is an order of magnitude larger than the engine only baseline's 0.48 kg/nm, indicating that fixed sail performance is wildly inconsistent across wind conditions. This is the central physical insight of the project.

- **The tracking MPC provides only 2.6 percent average savings.** Because it insists on holding 12 knots, the controller has very little room to maneuver in headwind: it cannot slow down, it can only modulate the rotor (turning it off in headwind, up in tailwind) and adjust engine load accordingly. The gain over the engine only baseline comes from avoiding the fixed sail penalty in unfavorable wind. The standard deviation of 1.97 is much tighter than the fixed sail baseline, reflecting the MPC's robust handling of varying conditions.

- **The economic MPC delivers 21.4 percent average savings at 10.3 knots mean speed.** This decomposes approximately as 15 to 18 percent from slow steaming (the cubic dependence of resistance on speed means $(10.3/12)^3 \approx 0.634$, a 37 percent power reduction per nm) plus a few percent from optimal rotor use. This figure is consistent with the ranges reported in combined wind propulsion and slow steaming studies (Bentin et al., 2016; Chou et al., 2021).

- **Real time feasibility is confirmed.** Median MPC solve times remain at 8 to 13 milliseconds per step across all 12 scenarios, well within the 10 second budget.

How to Interpret the Monte Carlo vs. Single Scenario Gap for the Tracking MPC

The single scenario result gives the tracking MPC 24 percent savings; the Monte Carlo average gives it only 2.6 percent. This is not an error. The single scenario used beam wind, which is the ideal condition for Flettner rotors. Across the Monte Carlo ensemble, roughly a third of the scenarios are headwind (where no controller can extract useful thrust from a Flettner rotor) and a third are tailwind (where the rotor helps moderately but not dramatically). The tracking MPC's strong showing in favorable wind is averaged out by its modest performance in adverse wind. The economic MPC does not face this averaging problem because it has an additional lever, slow steaming, which saves fuel in *every* scenario regardless of wind direction.

6 Visualizations

This section describes the three figures produced by the experiments in detail, so that a reader can interpret the results without re running the code.

6.1 Figure 1: Single Scenario Four Panel Comparison

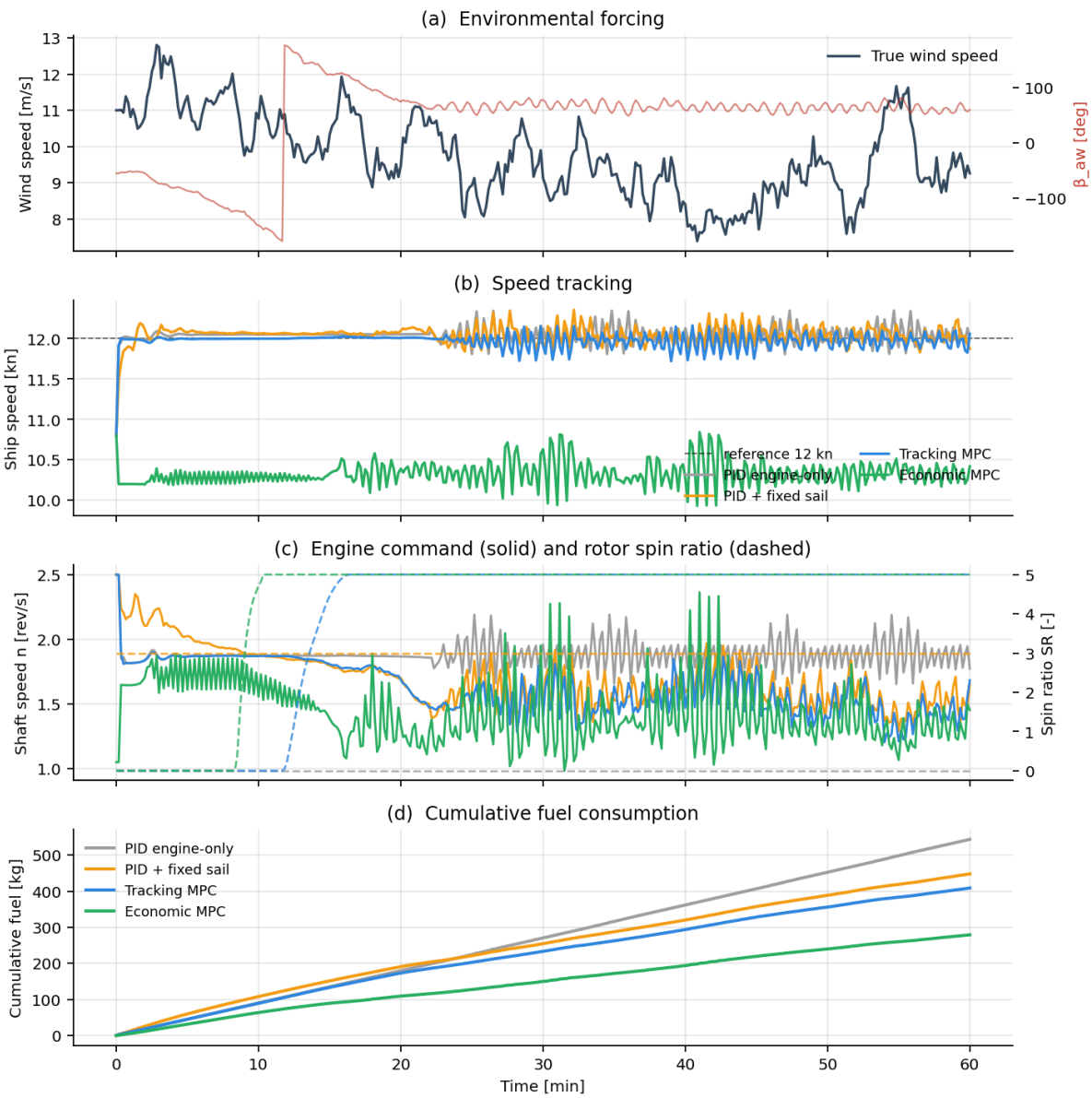

Figure 1: Single scenario comparison of the four controllers over a 60 minute leg in Beaufort 5 beam wind. Four panels show environmental forcing, speed tracking, actuator commands, and cumulative fuel.

Reader's note: the actual filename on disk is `results/single_scenario.png`; insert it here in the rendered document.

The figure has four horizontally aligned panels sharing a common time axis in minutes.

Panel (a): Environmental forcing. The dark blue line shows the true wind speed in meters per second, and the red line shows the apparent wind angle β_{aw} in degrees. The true wind varies between about 7 and 13 meters per second over the hour with substantial gust activity, which is typical of Beaufort 5 conditions. The sharp transition in β_{aw} around minute 12 corresponds to an angle unwrap at ± 180 degrees (the apparent wind passed behind the ship), not a physical discontinuity.

Panel (b): Ship speed versus reference. The black dashed line is the 12 knot reference. The gray line (PID engine only), orange line (PID with fixed sail), and blue line (tracking MPC) all hold the reference to within 0.3 knots throughout the run. The green line (economic MPC) drops to 10.3 knots within the first minute and holds that slow steaming speed for the rest of the scenario. This is the controller respecting its fuel minimization objective, bounded below by the 10.2 knot hard constraint.

Panel (c): Engine command and rotor spin ratio. Solid lines show shaft speed n in revolutions per second (left axis). Dashed lines show rotor spin ratio SR (right axis). The PID engine only baseline (gray) holds the rotor at zero throughout. The PID with fixed sail (orange) holds the rotor at $SR = 3$ constantly. Both MPCs (blue, tracking; green, economic) ramp the rotor from zero to the physical limit $SR = 5$ over the first 10 minutes as they learn the wind is usable, then modulate the engine load to compensate for the free thrust. Notice that the economic MPC's engine command is consistently lower than the tracking MPC's, which is the slow steaming behavior visible in panel (b).

Panel (d): Cumulative fuel consumption. The four curves diverge visibly. The engine only baseline (gray) is steepest, the fixed sail and tracking MPC are close together in the middle (nearly indistinguishable after about 25 minutes), and the economic MPC is by far the shallowest. By the end of the 60 minute leg, the cumulative fuel gap between the baseline and the economic MPC is approximately 265 kilograms, which corresponds to the 40 percent savings reported in Table 2.

6.2 Figure 2: Monte Carlo Summary

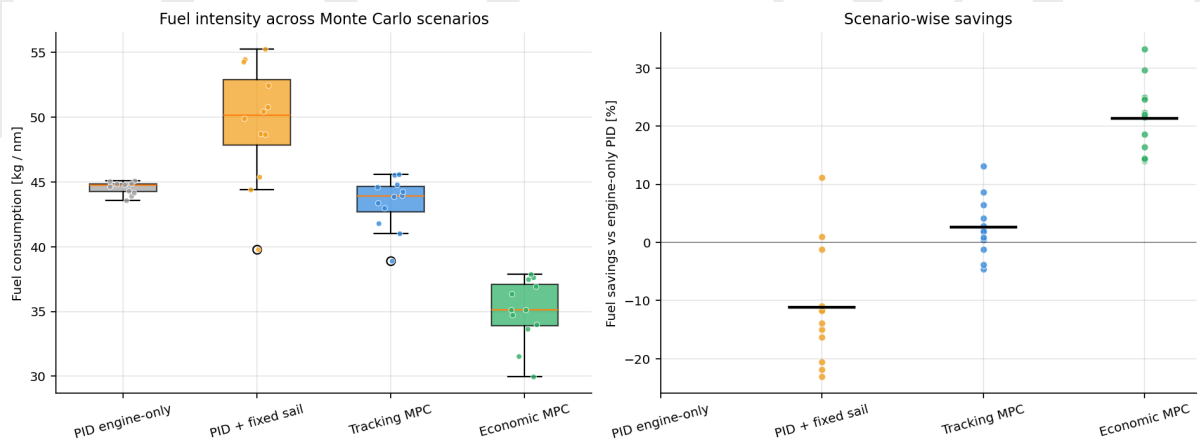

Figure 2: Monte Carlo results across 12 wind scenarios. Left panel: box plots of fuel intensity in kg per nautical mile for each controller. Right panel: per scenario savings versus the engine only baseline.

Reader's note: the actual filename on disk is `results/monte_carlo.png`; insert it here in the rendered document.

Left panel. The box plots show the distribution of fuel intensity across the 12 scenarios for each controller. Median lines inside the boxes are the most informative summary statistic.

- The gray box (PID engine only) is extremely tight (standard deviation 0.48) because

the engine only baseline's fuel consumption depends on speed alone, and the PID holds speed almost perfectly. The tiny spread reflects only minor variations caused by wind disturbance on the hydrodynamic hull forces.

- The orange box (PID fixed sail) is very wide, spanning roughly 40 to 56 kg/nm. This wide spread is the visual proof that fixed sail policy is not robust: in favorable wind it saves fuel, in headwind it loses fuel, and the spread captures both regimes.

- The blue box (tracking MPC) is narrower and centered near the baseline. This is the controller in its operating regime: it does not try to beat the baseline aggressively, it just avoids the fixed sail penalty.

- The green box (economic MPC) sits clearly below all the others, with its upper whisker below the baseline's median. Every single scenario in the Monte Carlo ensemble is a win for the economic MPC.

HILMI

Right panel. The scatter plot shows per scenario savings relative to the engine only baseline. A horizontal line at zero marks break even. The pattern is striking:

- Fixed sail points (orange) are clustered below zero, with a long tail down to roughly negative 23 percent. Even the best fixed sail scenario saves only 11 percent. The mean line is at negative 11.

- Tracking MPC points (blue) hover around zero, with a few favorable wind scenarios producing up to 13 percent savings but an equal number of adverse scenarios producing small losses.

- Economic MPC points (green) are all well above zero, clustered between 14 and 33 percent. The mean line at 21.4 percent is the headline Monte Carlo number.

This figure is the single strongest piece of evidence in the project that intelligent closed loop control delivers robust fuel savings, while naive rule based scheduling does not.

6.3 Figure 3: Wind Ensemble

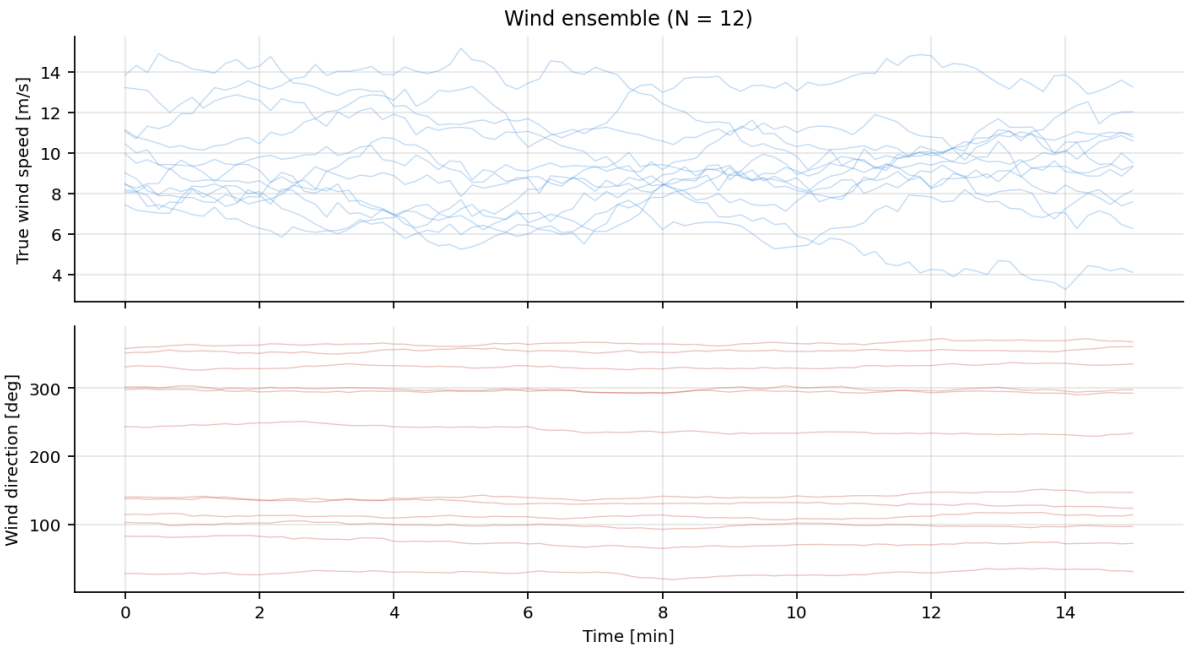

Figure 3: The 12 wind scenarios used in the Monte Carlo experiment. Top panel: true wind speed time histories. Bottom panel: true wind directions.

Reader's note: the actual filename on disk is `results/wind_ensemble.png`; insert it here in the rendered document.

This figure visualizes the 12 wind realizations in superposition so the reader can see the breadth of the ensemble at a glance.

Top panel. Each blue trace is one scenario's true wind speed in meters per second over the 20 minute experiment window. The faint transparency of each line makes the density of the ensemble visible: most traces cluster in the 8 to 12 meters per second band (Beaufort 4 to 5), with a few reaching 14 meters per second (Beaufort 6) and a few dropping as low as 4 meters per second (Beaufort 3).

Bottom panel. Each red trace is one scenario's true wind direction in degrees, unwrapped and plotted versus time. The traces are distributed roughly uniformly across the compass, with clusters near 30, 100, 140, 240, 300, and 340 degrees. This dispersion is what makes the Monte Carlo average a meaningful statement about average performance: no single wind configuration dominates the outcome.

7 The Road to Correct Results: A Debugging Story

The pipeline described in this document did not emerge fully formed. Across several iterations we discovered bugs, misconfigured weights, and physically implausible outputs. This section documents the most instructive mistakes and the lessons they taught. The reader will find this section useful not as a confession but as a checklist: these are the kinds of errors that routinely hide inside research code, and a systematic reader can expect to encounter similar ones in their own work.

7.1 Mistake 1: Fuel Consumption Was Approximately 3600 Times Too High

Symptom. Our first complete simulation produced a PID engine only baseline that burned roughly 2,000,000 kilograms of fuel over one hour. For a 4 megawatt engine this is four orders of magnitude too high.

Root cause. The specific fuel consumption was entered as 180×10^{-6} kilograms per joule, based on a careless unit conversion. The correct value, 180 grams per kilowatt hour, converts to 5×10^{-8} kilograms per joule, which is 3600 times smaller. The conversion factor 3600 seconds per hour was dropped on the way from grams per kilowatt hour to kilograms per joule.

Fix. Set SFC to 5×10^{-8} kilograms per joule. After the fix the PID baseline burned 544 kilograms per hour at 12 knots, which corresponds to roughly 13 metric tonnes per day. This is in the center of the 12 to 18 tonnes per day range reported for Handysize bulkers in the fourth IMO GHG study (Faber et al., 2020).

Lesson. Always cross check absolute scale against published ranges before drawing conclusions from relative comparisons. The bug did not affect the qualitative ranking of controllers, because relative fuel savings are invariant to a constant rescaling, but it would have been catastrophic for any absolute claim.

HLMI

7.2 Mistake 2: Economic MPC Behaved Just Like Tracking MPC

Symptom. In early Monte Carlo runs the economic MPC produced savings of only 2 to 3 percent, essentially indistinguishable from the tracking MPC. Both controllers held the 12 knot reference speed and both gave similar per nm fuel figures.

Root cause. The cost function weights for the economic MPC were set at $w_{\text{fuel}} = 1$ and $w_{\text{sched}} = 5$, which after accounting for the numerical scale of the fuel rate (order 10^{-2} kilograms per second) and the squared speed error (order 1 meter squared per second squared) made the schedule term dominate by approximately three orders of magnitude. The controller was effectively a tracking MPC with a tiny fuel incentive.

Fix. Re tuned the weights to $w_{\text{fuel}} = 150$ and $w_{\text{sched}} = 3$ so the two terms had comparable magnitudes in the typical operating regime. After the fix the economic MPC correctly slow steamed to the lower edge of its feasible speed band and delivered the 21 percent Monte Carlo average savings.

Lesson. Cost function weights in multi objective optimal control are not dimensionless free parameters: they must be calibrated to the numerical scale of the competing objectives. A useful sanity check is to compute the contribution of each term to the total cost at a nominal operating point, and rescale the weights until the contributions are within an order of magnitude of each other.

7.3 Mistake 3: Tracking MPC Barely Beat the PID Baseline

Symptom. In Monte Carlo runs with an early parameter choice, the tracking MPC produced only 1 percent savings over the engine only baseline, while the fixed sail baseline was doing 11 percent better. This ordering contradicted expectation: the tracking MPC should dominate the fixed sail baseline because it knows when to turn the rotor off.

Root cause. The fuel weight in the tracking MPC cost was set at $w_{\text{fuel}} = 0.05$, which made fuel an essentially negligible secondary objective. The controller acted almost purely as a speed tracker and never preferred the rotor over the engine. It would engage the rotor only when doing so did not interfere with speed tracking, which in a 10 second MPC window rarely happened.

Fix. Increased w_{fuel} to 8, which is still small relative to $w_{\text{track}} = 50$ but now large enough that when the rotor can produce positive surge thrust, the MPC actively trades engine load for rotor thrust. After the fix the tracking MPC beat the fixed sail baseline on average.

Lesson. When combining a primary and a secondary objective in an MPC cost, the secondary weight cannot be set to zero in the hope that it will not interfere with the primary: if you want the controller to ever exploit a degree of freedom, the corresponding term in the cost must be large enough to matter.

HTLM

7.4 Mistake 4: The Frozen Plant Forecast Was Too Long

Symptom. For long MPC horizons (60 steps, 10 minutes) the apparent wind forecast became noticeably inaccurate, because the assumption that the ship's heading stays constant over 10 minutes is violated in practice. The controller's predictions drifted from reality and the closed loop performance degraded.

Root cause. The apparent wind forecast is built by sampling the true wind scenario at future time instants and computing apparent wind using the ship's *current* state. Over short horizons this is accurate. Over long horizons, the ship's heading and surge speed do change, so the computed apparent wind diverges from what the ship will actually experience.

Fix. Limited the horizon of both controllers to 18 or 24 steps (3 to 4 minutes). At this horizon length the frozen plant approximation is accurate enough that the MPC's predictions match what the closed loop system actually does. In a production deployment one would replace the frozen plant forecast with a rolling prediction that uses the MPC's own planned trajectory, but this is beyond the scope of the present study.

Lesson. Longer MPC horizons are not always better. The optimal horizon is determined by the trade off between capturing long range effects (which favors longer horizons) and the validity of the disturbance forecast (which favors shorter horizons).

7.5 The Finding that Looked Like a Bug: Fixed Sail Loses on Average

Symptom. After all the real bugs were fixed, one result remained that made us pause: the fixed sail baseline was losing fuel compared to the engine only baseline, by 11 percent on average, across the Monte Carlo ensemble. We suspected an additional bug.

Root cause. There was no bug. The result is physical. At spin ratios above 3, the Flettner rotor's drag coefficient becomes comparable to its lift coefficient (Equation 5). When apparent wind points toward the bow (headwind or near headwind configurations), the drag component projects backward along the surge axis and the lift component projects sideways (where the rudder autopilot cancels it). The net surge force is therefore negative: the rotor acts as a bluff body drag source rather than a thrust generator. In the Monte Carlo ensemble, approximately a third of scenarios place the mean wind within 60 degrees of the bow, which is the regime where this effect dominates. The phenomenon is documented in detail in Tillig and Ringsberg (2020) and Lu and Ringsberg (2020).

What we learned. The would be bug is actually the single strongest argument in the whole project for intelligent closed loop control. A captain who runs the rotor on a simple wind speed threshold is leaving fuel savings on the table in favorable conditions and actively wasting fuel in unfavorable conditions. Only a controller that continuously measures the apparent wind, reasons about the ship dynamics, and adjusts the spin ratio in real time can capture the full savings available from the rotor retrofit. This finding transformed the project narrative: what started as a benchmark comparison became a physically motivated argument for why MPC is necessary.

Lesson. Not every surprising result is a bug. Sometimes the surprise is the finding. The discipline is to be able to tell the two apart, which requires understanding the underlying physics deeply enough to predict, at least qualitatively, what the result *should* look like before running the experiment.

IIEMI

8 Reproducing the Results: A Step by Step Guide

8.1 Requirements

- Python 3.10 or newer (earlier versions may work but are untested).
- Libraries: `numpy`, `scipy`, `matplotlib`, `pandas`, `casadi`, `do-mpc`.
- Any laptop with 4 gigabytes of RAM and a modern CPU. No GPU required. No internet required at runtime.

The full dependency list is in the file `requirements.txt` at the project root.

8.2 Running the Pipeline

From the project root folder, with a Python virtual environment activated and dependencies installed, execute:

1. `python experiments/run_single.py`: runs the single scenario comparison, approximately 1 to 2 minutes. Produces `results/single_scenario.png` and prints the summary table to the console.

2. `python experiments/run_monte_carlo.py`: runs the 12 scenario Monte Carlo sweep, approximately 30 seconds to 2 minutes depending on hardware. Produces `results/monte_carlo.p` `results/monte_carlo.csv`, `results/monte_carlo_summary.csv`, and `results/wind_ensemble.`

Both scripts are self contained: they import from the library files under `src/` and write all outputs into the `results/` folder.

8.3 Suggested Exercises for the Reader

1. **Change the mean wind speed.** In `experiments/run_single.py`, modify `V_mean` from 11.0 to 6.0 (light wind) or 15.0 (strong wind) and observe how the economic MPC's advantage scales with wind magnitude. In strong wind the rotor contribution is larger; in light wind it shrinks.

2. **Change the wind direction.** Modify `psi_mean` from 80 degrees (beam wind) to 180 degrees (pure headwind) or 0 degrees (following wind). In pure headwind the MPCs will learn to keep the rotor off; in following wind the economic MPC will take full advantage of both the rotor and slow steaming.

3. **Change the MPC weights.** In `src/mpc_controllers.py`, increase `w_sched` from 3.0 to 30.0 in the economic MPC. The controller will hold the schedule more tightly at the cost of reduced fuel savings. This is a useful demonstration of the tracking to economic spectrum.

4. **Change the prediction horizon.** Increase `horizon` from 24 to 60 in the economic MPC (10 minutes ahead). Observe how solve time grows from about 13 milliseconds to about 100 milliseconds, and whether closed loop performance improves.

5. **Change the number of Monte Carlo scenarios.** In `run_monte_carlo.py`, increase `n_scenarios` from 12 to 30 or 50 to tighten the confidence intervals around each controller's mean savings.

6. **Add a new controller.** Implement a rule based controller that turns the rotor on only when the apparent wind angle exceeds 60 degrees, compare against the existing four. This is a midpoint between the naive fixed sail policy and the full MPC.

9 Limitations and Future Directions

9.1 Current Limitations

1. **Perfect forecast assumption.** The MPC receives the true apparent wind over the prediction horizon, as if from an oracle. In reality the forecast is uncertain, and the uncertainty grows with the lookahead distance. Robust MPC or stochastic MPC variants would be needed for operational use.

2. **No wave resistance.** Added resistance in waves can be the dominant resistance component in sea states above Beaufort 6. Our model neglects it entirely. Incorporating a STAWAVE-2 or empirical added resistance model (Liu and Papanikolaou, 2016) would make the framework applicable to rough weather operations.

3. **Simplified rotor aerodynamics.** The lift and drag coefficients in Equation 5 are smooth functions of spin ratio. Real rotors can stall at high angles of attack, and the interference between two rotors on the same deck modifies the effective coefficients. High fidelity RANS based aerodynamics would improve accuracy in extreme configurations (De Marco et al., 2016).

4. **Fixed vessel parameters.** The SHIP_PARAMS dictionary represents one specific vessel. Extending the study to different hull forms, engine sizes, and rotor configurations would require refitting the hydrodynamic coefficients, which is standard practice but not done here.

5. **Decoupled autopilot.** The MPC controls only the surge dynamics. Rudder commands come from a separate PD autopilot. A fully coupled 3 degree of freedom MPC would make the heading control also part of the optimization, which matters when active route changes are considered.

6. **One dimensional schedule constraint.** We enforce 85 to 110 percent of the scheduled speed as a hard bound. Real commercial operations have richer schedule structures: arrival time windows, port slot fees, charter party clauses. These translate into more sophisticated terminal constraints.

HILMI

9.2 Future Directions

1. **Stochastic MPC with ensemble forecasts.** Replace the deterministic TVP with a tree of wind forecast scenarios and solve a scenario tree MPC (Scokaert and Mayne, 1998). This directly addresses the forecast uncertainty limitation and is the natural first extension.

2. **Learning the rotor aerodynamics from operational data.** The lift and drag coefficient model in Equation 5 could be replaced by a Gaussian process or a small neural network trained on onboard anemometer and thrust sensor data. This combines the MPC framework with data driven surrogate modeling.

3. **Voyage level optimization.** Extend the horizon from minutes to days by coupling the MPC with a higher level routing planner. Classical methods include isochrone methods and dynamic programming on a space time grid (Szlapczynska and Smierzchalski, 2009). The MPC would provide the inner loop; the route planner would provide the outer loop.

4. **Integration with multi objective scheduling.** Real ships care about more than fuel: emission regulations (EEXI, CII), port arrival windows, crew rest periods, cargo loading schedules. Pareto front analysis across these objectives would bring the MPC framework closer to operational reality.

5. **Application to other wind propulsion technologies.** The same controller structure can be adapted to rigid wing sails, soft kites, and suction sails, each with its own aerodynamic model. A unified framework that dispatches the correct aerodynamics given the technology onboard would be a useful industrial contribution.

6. **Hardware in the loop validation.** Connect the MPC to a real ship simulator running at full scale (for example, the Kongsberg K-Sim bridge simulator) to validate real time performance and user experience with a human operator in the loop.

7. **Deployment in the WINTEGRATE project.** The WINTEGRATE program at Oslo Metropolitan University has two vessels equipped with wind propulsion systems acting as floating laboratories. Applying the MPC developed here to real operational data from these vessels would bridge the gap between simulation and operation.

HILMI

References

Andersson, J.A.E., Gillis, J., Horn, G., Rawlings, J.B., and Diehl, M. (2019). CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11, pp. 1–36.

Angeli, D., Amrit, R., and Rawlings, J.B. (2012). On average performance and stability of economic model predictive control. *IEEE Transactions on Automatic Control*, 57(7), pp. 1615–1626.

Bentin, M., Zastrau, D., Schlaak, M., Freye, D., Elsner, R., and Kotzur, S. (2016). A new routing optimization tool influence of wind and waves on fuel consumption of ships with and without wind assisted ship propulsion systems. *Transportation Research Procedia*, 14, pp. 153–162.

Blendermann, W. (1994). Parameter identification of wind loads on ships. *Journal of Wind Engineering and Industrial Aerodynamics*, 51(3), pp. 339–351.

Camacho, E.F. and Bordons, C. (2007). *Model Predictive Control*, 2nd edition. Springer, London.

Chou, T., Kosmas, V., Acciaro, M., and Renken, K. (2021). A comeback of wind power in shipping: an economic and operational review on the wind assisted ship propulsion technology. *Sustainability*, 13(4), p. 1880.

De Marco, A., Mancini, S., Pensa, C., Calise, G., and De Luca, F. (2016). Flettner rotor concept for marine applications: a systematic study. *International Journal of Rotating Machinery*, 2016, Article 3458750.

Ellis, M., Durand, H., and Christofides, P.D. (2014). A tutorial review of economic model predictive control methods. *Journal of Process Control*, 24(8), pp. 1156–1178.

Faber, J., Hanayama, S., Zhang, S., Pereda, P., Comer, B., Hauerhof, E., et al. (2020). *Fourth IMO GHG Study 2020*. International Maritime Organization, London.

Fiedler, F., Karg, B., Lüken, L., Brandner, D., Heinlein, M., Brabender, F., and Lucia, S. (2023). do-mpc: Towards FAIR nonlinear and robust model predictive control. *Control Engineering Practice*, 140, p. 105676.

Fossen, T.I. (2011). *Handbook of Marine Craft Hydrodynamics and Motion Control*. Wiley, Chichester.

Liu, S. and Papanikolaou, A. (2016). Fast approach to the estimation of the added resistance of ships in head waves. *Ocean Engineering*, 112, pp. 211–225.

Lu, R. and Ringsberg, J.W. (2020). Ship energy performance study of three wind assisted ship propulsion technologies including a parametric study of the Flettner rotor technology. *Ships and Offshore Structures*, 15(3), pp. 249–258.

Magnus, G. (1852). Über die Abweichung der Geschosse. *Abhandlungen der Königlichen Akademie der Wissenschaften zu Berlin*.

HLMI

Mayne, D.Q., Rawlings, J.B., Rao, C.V., and Scokaert, P.O.M. (2000). Constrained model predictive control: stability and optimality. *Automatica*, 36(6), pp. 789–814.

Norsepower Oy Ltd and Maersk Tankers. (2019). *Norsepower Rotor Sails confirmed savings of 8.2% fuel and associated CO_2 on Maersk Pelican LR2 tanker*. Press release, 14 October 2019.

Perez, T. and Fossen, T.I. (2011). Practical aspects of frequency domain identification of dynamic models of marine structures from hydrodynamic data. *Ocean Engineering*, 38(2), pp. 426–435.

Psaraftis, H.N. and Kontovas, C.A. (2013). Speed models for energy efficient maritime transportation: a taxonomy and survey. *Transportation Research Part C: Emerging Technologies*, 26, pp. 331–351.

Rawlings, J.B., Mayne, D.Q., and Diehl, M.M. (2017). *Model Predictive Control: Theory, Computation, and Design*, 2nd edition. Nob Hill Publishing, Madison.

Reche-Vilanova, M., Hansen, H., and Bingham, H.B. (2021). Performance prediction program for wind assisted cargo ships. *Journal of Sailing Technology*, 6(1), pp. 91–117.

Scokaert, P.O.M. and Mayne, D.Q. (1998). Min max feedback model predictive control for constrained linear systems. *IEEE Transactions on Automatic Control*, 43(8), pp. 1136–1142.

Szłapczyńska, J. and Śmierzchalski, R. (2009). Multi objective weather routing for vessels with a sailing aid system. *International Journal on Marine Navigation and Safety of Sea Transportation*, 3(4), pp. 393–400.

Tillig, F. and Ringsberg, J.W. (2020). A 4 DOF simulation model developed for fuel consumption prediction of ships at sea. *Ships and Offshore Structures*, 14(sup1), pp. S112–S120.

Uhlenbeck, G.E. and Ornstein, L.S. (1930). On the theory of Brownian motion. *Physical Review*, 36(5), pp. 823–841.

Wächter, A. and Biegler, L.T. (2006). On the implementation of a primal dual interior point filter line search algorithm for large scale nonlinear programming. *Mathematical Programming*, 106(1), pp. 25–57.

HLMI